Cipher

Fluffy the Squirrel has a string S of lowercase English letters, and is trying to encode it according to a certain cipher.

The cypher can be described by two strings A and B, the i^{th} letter in string A is mapped to the i^{th} letter in string B. For example, if `A = "abcdefghijklmnopqrstuvwxyz"`, and `B = "zyxwvutsrqponmlkjihgfedcba"`, then `a` is mapped to `z`, `b` is mapped to `y` and so on.

Transforming the string just once is boring. So additionally, given a positive integer K, Fluffy would like to repeat the process K times instead. But now she is tired, and asks for your help. Can you help her to get the final string?

Constraints

- Length of $S \leq 10^5$,
- $1 \leq K \leq 10^5$,
- S consists of only lowercase English letters,
- A, B is a permutation of lowercase English letters.

Sample Input

```
S = "ctej"
K = 2
A = "zyxwvutsrqponmlkjihgfedcba"
B = "cbafedihglkjonmrqputsxwvzy"
```

Sample Output

```
epal
```

Explanation:

Consider the first letter of S, `c`.
After the first transformation, it changes from `c` to `v`.
After the second transformation, it changes from `v` to `e`, which is the first letter of the answer.
The process is similar for the rest of the letters.

Solution

The most straightforward way is to actually simulate the process of shifting. But this can be quite inefficient, as it takes roughly $|S| * K$ steps to complete ($|S|$ means length of S), because we have to transform K times for each character in S. This will be good enough for small cases for case 1 to 6. However, more optimisation needs to be done to compute quickly for large cases for case 7 to 10.

Key observation:
Once we know the result for one letter, there is no need to simulate it again! For example, if we know that 'a' changes to 'z' after shifting K times, then we simply need to go through S once to change all 'a' to 'z'.

With this knowledge, there are a lot of ways we can go on from here. The first step would be to construct the final mapping. The alphabet size is small enough that it can even be computed by hand (draw out a network with the letters, and determine the final destination based on K). Or more practically, we can just simulate once shifting K times for each letter 'a' to 'z', and store the result somewhere. You can do using two strings similar to A and B, or more elegantly, using a map (known as dictionary in Python). This will take at most roughly K steps.

After that, you simply need to go through S and transform each letter only once according to the final mapping. This will take roughly $|S|$ steps.

So in the end, this only requires $K + |S|$ steps, a significant improvement from taking $|S| * K$ steps, and this will be good enough to solve case 7 to 10 quickly.

Here is an implementation in Python for the problem.

Implementation

```
S = "ctej"
K = 2
A = "zyxwvutsrqponmlkjihgfedcba"
B = "cbafedihglkjonmrqputsxwvzy"

### Construct the initial mapping
cipher = dict()

for i in range(26):
    cipher[A[i]] = B[i]

### Construct the final mapping by simulation
final_mapping = dict()

for c in "abcdefghijklmnopqrstuvwxyz":
    dest = c
    rem = K
    while rem > 0:
        dest = cipher[dest]
        rem -= 1
    final_mapping[c] = dest

### Construct the answer
ans = ""
for c in S:
    ans += final_mapping[c]

print(ans)
```